

WORKSHEET PRAKTIKUM

Mata Kuliah : Strategi Algoritma Topik : Algoritma
Brute Force Nama : Azalia Anindya Malya NIM
: 247006111040 Kelas : B
Dosen Pengampu : **Ir. Randi Rizal, Ph.D., IPM.**

PRAKTIKUM 1 — Pencarian String (Pattern Matching)

1. Analisis Masalah

Teks : ALGORITMASTRATEGIALGORITMA

Pattern : RIT

Panjang teks (n) = 27

Panjang pattern (m) = 3

Jumlah kemungkinan posisi = 25

2. Pseudocode Brute Force

```
For i = 0 to n-m:      match = true      For
j = 0 to m-1:          bandingkan text[i+j]
dan pattern[j]         jika tidak sama →
break                 jika match → simpan indeks
```

3. Implementasi Program

```
text =
"ALGORITMASTRATEGIALGORITMA"
pattern = "RIT"
```

```
n = len(text) m
= len(pattern)
comparisons = 0
positions = []
```

```
for i in range(n - m + 1):
    match = True    for j in
range(m):          comparisons
+= 1               if text[i + j] !=
pattern[j]:
    match = False
    break
if match:
    positions.append(i)
```

```
print("Indeks:", positions)
print("Jumlah perbandingan:", comparisons)
```

4. Hasil Eksekusi & Screen Shoot

```
=====
TUGAS PRAKTIKUM ALGORITMA BRUTE FORCE
Nama : Azalia Anindya Malya
NIM : 247006111040
=====

=====
PRAKTIKUM 1 - STRING MATCHING
=====
Indeks kemunculan pattern: [4, 21]
Jumlah perbandingan karakter: 29
```

5. Analisis Kompleksitas

Worst-case time complexity: $O(nm)$

Penjelasan:

Algoritma brute force untuk pencarian string menggunakan dua loop bersarang:

- Loop luar berjalan sebanyak $(n - m + 1)$ kali.
- Loop dalam membandingkan maksimal m karakter setiap posisi.

Pada kondisi worst-case (misalnya semua karakter teks hampir sama dengan pattern kecuali karakter terakhir), maka setiap posisi akan melakukan perbandingan sebanyak m kali.

Sehingga total operasi perbandingan karakter dalam kondisi terburuk adalah:

$$(n-m+1) \times m$$

Secara notasi Big-O disederhanakan menjadi:

$$O(nm)$$

Hal ini menunjukkan bahwa kompleksitas bertumbuh secara linier terhadap panjang teks dan pattern.

6. Pertanyaan Reflektif

Kapan brute force tidak efisien?

- Brute force menjadi tidak efisien ketika:

- Panjang teks (n) sangat besar.
- Panjang pattern (m) cukup besar.
- Banyak karakter yang mirip sehingga hampir semua perbandingan dilakukan penuh.
- Digunakan pada data skala besar atau sistem real-time.

Karena kompleksitasnya $O(nm)$, waktu eksekusi meningkat secara signifikan jika ukuran input bertambah.

Mengapa brute force tetap relevan dalam strategi algoritma?

- Brute force tetap relevan karena:
 1. Mudah dipahami dan diimplementasikan.
 2. Menjamin menemukan solusi jika solusi memang ada.
 3. Menjadi dasar untuk memahami algoritma optimasi seperti:
 - KMP
 - Boyer-Moore
 4. Cocok untuk ukuran input kecil.
 5. Berguna sebagai pembanding untuk mengukur efisiensi algoritma lain.

PRAKTIKUM 2 — Subset Sum 1.

Analisis Masalah

Array A = {3,5,9,12} Target
= 17

Jumlah elemen (n) = 4

2. Pseudocode

```
Input: array A, target n
← panjang(A)

For i from 0 to 2^n - 1:
    sum ← 0    subset ← {}
    For j from 0 to n-1:
        If bit j dari i aktif:
            tambahkan A[j] ke subset
    sum += A[j]    Jika sum == target:
        tampilkan subset
```

3. Implementasi Program

```
A = [3, 5, 9, 12] target
= 17
n = len(A)
```

```
count = 0
```

```

solutions = []

for i in range(2**n):
    subset = []
    total = 0
    for j in range(n):
        if (i >> j) & 1:
            subset.append(A[j])
            total += A[j]
            count += 1
    if total == target:
        solutions.append(subset)

print("Total subset diperiksa:", count)
print("Subset yang memenuhi target:", solutions)

```

4. Hasil & Screen Shoot

```

=====
TUGAS PRAKTIKUM ALGORITMA BRUTE FORCE
Nama : Azalia Anindya Malya
NIM : 247006111040
=====

=====
PRAKTIKUM 2 - SUBSET SUM
=====
Total subset diperiksa: 16
Subset yang memenuhi target: [[3, 5, 9], [5, 12]]

```

5. Tabel Runtime

n	Jumlah Subset (2^n)	Runtime (ms)
4	16	0.0098
5	32	0.0253
6	64	0.0250

6. Analisis

Mengapa kompleksitasnya $O(2^n)$?

Kompleksitas algoritma Subset Sum brute force adalah **$O(2^n)$** karena algoritma tersebut mengeksplorasi seluruh kemungkinan subset dari suatu himpunan yang memiliki n elemen.

Untuk setiap elemen terdapat dua kemungkinan:

- Elemen dipilih

- Elemen tidak dipilih

Sehingga total kemungkinan kombinasi yang terbentuk adalah:

$$2^n$$

Karena algoritma memeriksa seluruh subset tersebut satu per satu untuk menghitung jumlahnya dan membandingkannya dengan target, maka jumlah operasi bertumbuh secara eksponensial terhadap n .

Dengan demikian, kompleksitas waktunya adalah:

$$O(2^n)$$

Jika $n = 20$, apa dampaknya?

Jika $n = 20$, maka jumlah subset yang harus diperiksa adalah:

$$2^{20} = 1.048.576 \cdot 2^{20} = 1.048.576 \cdot 2^{20} = 1.048.576$$

Artinya, algoritma harus memeriksa lebih dari 1 juta kombinasi.

Dampaknya:

- Waktu eksekusi meningkat drastis.
- Program menjadi jauh lebih lambat.
- Penggunaan CPU meningkat.
- Tidak efisien untuk data berukuran besar.

Karena pertumbuhan bersifat eksponensial, kenaikan nilai n sedikit saja akan menyebabkan lonjakan jumlah kombinasi yang sangat besar.

Sebagai contoh:

- $n = 10 \rightarrow 1.024$ subset
- $n = 20 \rightarrow 1.048.576$ subset
- $n = 30 \rightarrow 1.073.741.824$ subset

Hal ini menunjukkan bahwa brute force tidak cocok untuk skala besar tanpa optimasi tambahan.

PRAKTIKUM 3 — Traveling Salesman Problem (TSP Mini)

1. Analisis

Jumlah kota = 4

Permutasi rute = $(n-1)! = 3! = 6$

2. Pseudocode

Input: matriks jarak `dist[n][n]`

```

Fix kota awal = 0
cities ← {1,2,3,...,n-1}
min_distance ← tak hingga
best_route ← kosong
count ← 0

Untuk setiap permutasi P dari cities:
    route ← [0] + P + [0]
    total_distance ← 0

    Untuk i dari 0 sampai panjang(route)-2:
        total_distance ← total_distance +
        dist[route[i]][route[i+1]]
        count ← count + 1

    Jika total_distance < min_distance:
        min_distance ← total_distance
        best_route ← route

Tampilkan:
    jumlah rute diperiksa = count
    rute terpendek = best_route
    total jarak minimum = min_distance

```

3. Implementasi Program

import
itertools

```

dist = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]

```

```

cities = [1,2,3] # B,C,D

```

```

min_distance = float('inf')
best_route = None
count = 0

for perm in itertools.permutations(cities):
    route = [0] + list(perm) + [0]
    total = 0
    for i in range(len(route)-1):
        total += dist[route[i]][route[i+1]]
    count += 1
    if total < min_distance:
        min_distance = total
        best_route = route

print("Jumlah rute diperiksa:", count)
print("Rute terpendek:", best_route)
print("Total jarak minimum:", min_distance)

```

4. Hasil & Screen Shoot

```

=====
TUGAS PRAKTIKUM ALGORITMA BRUTE FORCE
Nama : Azalia Anindya Malya
NIM : 247006111040
=====

=====
PRAKTIKUM 3 - TSP MINI
=====
Jumlah rute diperiksa: 6
Rute terpendek: [0, 1, 3, 2, 0]
Total jarak minimum: 80

```

5. Kompleksitas

Time complexity = $O(n!)$

Jika kota = 10 $\rightarrow$ jumlah rute = $(10-1)! = 9! = 362.880$

PRAKTIKUM 4 — Password Brute Force Simulation

1. Analisis

Jumlah karakter alfabet = {a,b,c,1,2} Panjang password = 3

Total kombinasi = $|\Sigma|^L = 5^3 = 125$

2. Pseudocode

```
Input: alfabet  $\Sigma$ , panjang L, target
count  $\leftarrow$  0
ditemukan  $\leftarrow$  false

Untuk setiap kombinasi karakter sepanjang L
dari  $\Sigma$ :
    bentuk string guess
    count  $\leftarrow$  count + 1

    Jika guess = target:
        tampilkan "Password ditemukan"
    tampilkan "Percobaan ke-", count
    ditemukan  $\leftarrow$  true          berhenti

Jika tidak ditemukan:    tampilkan
"Password tidak ditemukan"

Tampilkan total kombinasi diperiksa = count
```

3. Hasil Eksperimen

```
=====
TUGAS PRAKTIKUM ALGORITMA BRUTE FORCE
Nama : Azalia Anindya Malya
NIM  : 247006111040
=====

=====
PRAKTIKUM 4 - PASSWORD BRUTE FORCE
=====
Password ditemukan pada percobaan ke: 41
Total kombinasi diperiksa: 41
```

4. Analisis Kompleksitas

Kompleksitas waktu = $O(|\Sigma|^L)$

Penjelasan pertumbuhan kompleksitas:

Kompleksitas waktu algoritma brute force password adalah:

$$O(|\Sigma|^L)$$

di mana:

- $|\Sigma|$ = jumlah karakter dalam alfabet
- L = panjang password

Pertumbuhan kompleksitas bersifat eksponensial, karena setiap penambahan 1 karakter pada panjang password akan mengalikan jumlah kemungkinan kombinasi dengan $|\Sigma|$.

TUGAS PRAKTIKUM

1. Ringkasan Konsep Brute Force

Brute Force adalah strategi penyelesaian masalah dengan cara mencoba semua kemungkinan solusi yang ada hingga menemukan solusi yang benar. Pendekatan ini tidak menggunakan optimasi atau heuristik, melainkan melakukan pencarian secara menyeluruh (exhaustive).

Metode ini sederhana, mudah diimplementasikan, dan menjamin solusi ditemukan jika memang ada. Namun, karena mengeksplorasi seluruh kemungkinan, brute force sering memiliki kompleksitas waktu yang tinggi seperti eksponensial atau faktorial.

2. Kompleksitas Waktu (Big-O)

No.	Studi Kasus	Kompleksitas
1.	String Matching	$O(nm)$
2.	Subset Sum	$O(2^n)$
3.	TSP	$O(n!)$
4.	Password	$O(L^N)$

3. Tabel Perbandingan Kandidat vs Runtime

Kasus	Jumlah Kandidat	Runtime
String Matching	25 posisi	Cepat (linear)
Subset Sum ($n=4$)	16 subset	Sangat cepat
TSP (4 kota)	6 rute	Cepat
Password ($L=3$)	125 kombinasi	Cepat

4. Kesimpulan

Kelebihan Brute Force:

1. Mudah dipahami dan diimplementasikan.
2. Menjamin menemukan solusi jika solusi ada.

Kelemahan Brute Force:

1. Tidak efisien untuk input besar.
2. Kompleksitas dapat tumbuh sangat cepat (eksponensial/faktorial).

SOAL REFLEKTIF

1. Mengapa brute force tetap penting?

Karena brute force merupakan dasar dari banyak strategi algoritma. Dengan memahami brute force, mahasiswa dapat memahami bagaimana algoritma bekerja sebelum dioptimalkan. Selain itu, brute force dapat digunakan sebagai pembanding untuk mengukur efisiensi algoritma yang lebih kompleks.

2. Dalam kondisi apa brute force lebih baik?

Brute force lebih baik digunakan ketika:

- Ukuran input kecil.
- Waktu pengembangan terbatas.
- Dibutuhkan solusi cepat dan sederhana.
- Masalah tidak terlalu kompleks.

3. Apa hubungan brute force dengan exhaustive search?

Brute force adalah bentuk dari exhaustive search. Keduanya samasama mengeksplorasi seluruh ruang solusi tanpa mengabaikan kemungkinan apa pun. Exhaustive search adalah istilah umum, sedangkan brute force adalah implementasi langsung dari pendekatan tersebut.